

Good — RiskRadar already has real technical bones (ontology, KG, LLM extraction, IOGP rule engine, a scoring formula). That's exactly the problem: it's a *competent* architecture, and competent is what gets a "nice dashboard, next team please." The four ideas below are chosen specifically because none of them are extensions of what you already have — each one imports a genuinely different discipline's *core methodology*, not just its vocabulary, and each is buildable from text + metadata alone.

1. SIF-ATT&CK™ — The Precursor Tactics & Techniques Matrix

The Core Novelty: Replace your flat hazard taxonomy with a MITRE ATT&CK-style matrix — the exact framework SOC teams use for cyber threat hunting — where precursors are organized as **Tactics** (the safety-defeat objective, e.g. "Defense Evasion" = bypassing a control) and **Techniques** (the specific observed method, e.g. "T1: Isolation Assumed, Not Verified"). This turns your taxonomy from a list into a *coverage map*.

How It Works (Technical Flow):

1. Re-model `taxonomy/hazards.yaml` + `barriers.yaml` as a 2-axis matrix: 6–8 Tactics × 25–40 Techniques (built directly from precursor examples you've already catalogued).
2. Extraction gets a RAG-grounded technique classifier — same LLM call, one more field, few-shot examples per technique.
3. Every processed report becomes a "Technique Occurrence" event, aggregated into a live heatmap (literally the ATT&CK Navigator, re-skinned for barriers).
4. **Technique Chaining:** flag reports where ≥2 techniques co-occur (e.g. "Permit Backdated" + "Isolation Assumed") as a **Campaign** — a compound signal your current per-report scoring can't see, because it treats each field independently.

Why Judges/OIL Will Love It: Any technical judge with a security background will clock the reference in two seconds — it signals "this team thinks in systems, not scores." And it gives OIL leadership a genuinely new artifact: "*We currently detect 34 of 50 known precursor techniques — here are our 16 blind spots.*" That's a maturity narrative, not a dashboard. VelocityEHS and Cority classify severity; neither publishes anything resembling a detection-coverage map. That's your defensible white space.

Hackathon Implementation Plan: Half a day to hand-curate the matrix (you're mostly relabeling work you've already done). The heatmap is a static grouped-count grid — no ML required for the viz itself. Technique chaining is a co-occurrence rule over fields you already extract. Zero new infrastructure.

2. Precursor Epidemiology — The Outbreak Detection Engine

The Core Novelty: Stop treating "3 similar reports in 45 days" as your pattern-detection ceiling. Borrow **spatial-temporal scan statistics** (the Kulldorff-class algorithm used for disease-outbreak and crime-hotspot detection) for statistically rigorous clustering, and borrow **fraud-ring entity resolution** from FinTech to track *contractors and shift crews* as graph nodes — not just sites — so you can catch a "Typhoid Mary" pattern: the same contractor showing up as the common thread across precursors at three unrelated sites.

How It Works (Technical Flow):

1. Add two cheap metadata fields to intake: `crew_id`, `contractor_name` (still pure text, no IoT).
2. Run a discrete Poisson scan statistic over (site × time-window) per hazard category — output isn't "3 in 45 days," it's " $p = 0.03$, $4.2\times$ this hazard's baseline rate at this site."
3. Build a lightweight entity graph in Postgres: nodes = {site, hazard, technique, contractor, crew}, edges = co-occurrence. Run a common-neighbor query to surface the highest-centrality entity across your high-SIF-potential subgraph — the exact technique AML investigators use to find a shared mule account across "unrelated" fraud cases.
4. Ship two new artifacts: a **statistical-significance badge** replacing the raw count, and an **Entity Radar** ranking contractors by risk-centrality, independent of site.

Why Judges/OIL Will Love It: This converts a PS-mandated feature (pattern discovery) from a heuristic into an actual inferential claim — a serious credibility jump for any quant-literate judge. And it answers a real organizational question none of your competitors touch: *is this a site problem, or a training problem that's following one contractor around?* That's a genuinely non-obvious insight class.

Hackathon Implementation Plan: The scan statistic is ~50 lines of NumPy — no need for full SaTScan software. The entity graph reuses your existing `patterns` table with a new `pattern_type = 'entity_cluster'`. Engineer two seed-dataset rows (one obvious outbreak, one obvious shared-contractor case) to make the demo land.

3. The Digital Bow-Tie — Probabilistic Kill-Chain Simulation

The Core Novelty: Your static weighted formula ($S = 0.2E + 0.2X + \dots$) is honest but it's still a point estimate — and every judge who's done process safety will ask "where do the weights come from?" Answer that by running an actual **Monte Carlo simulation over an auto-constructed Bow-Tie**, using generic failure-rate priors from **OREDA** (Offshore & Onshore Reliability Data — the real, public reliability database used in O&G Quantitative

Risk Assessment). This is Probabilistic Risk Assessment, the exact methodology nuclear and aerospace safety engineering runs on — applied live to NLP-extracted evidence instead of engineering drawings.

How It Works (Technical Flow):

1. Your existing PrecursorChain (Hazard → Barrier → Barrier Status) *is* a bow-tie's left half — you just haven't run it as one yet.
2. Map each `barrier_status` to a generic, publicly-documented probability-of-failure-on-demand range (e.g. manual isolation-verification failure has a published generic PFD in QRA literature) — never an invented number.
3. Run 5,000–10,000 Monte Carlo trials sampling barrier failure + base energy-release likelihood.
4. Output a distribution: *"72% negligible, 21% recordable, 7% SIF-level escalation if repeated — based on generic industry reliability data, illustrative not OIL-specific"* — shown as a probability band alongside your existing score, not replacing it.

Why Judges/OIL Will Love It: Nothing signals "we understand real industrial risk engineering" like citing OREDA and running an actual simulation instead of a linear weighted sum. This is your single strongest card for the Oil & Gas engineer on the judging panel specifically. It also turns "how did you pick these weights?" from a defensive question into your best answer of the round.

Hackathon Implementation Plan: One `numpy` module, under 100 lines. A lookup table of 8–10 generic PFDs, clearly cited and labeled illustrative. One new JSON column on `sif_assessments` to store the distribution. No new database tables, no new services.

4. The Cassandra Protocol — Calibration-Gamed Human Review

The Core Novelty: You already know automation bias is your scariest failure mode (it's in your own risk register). Don't just say "human stays in the loop" — borrow **cognitive forcing functions** from clinical AI-assisted diagnosis (radiology's blind-first second-reader studies) and **calibration training** from Philip Tetlock's superforecasting research. Reviewers commit a judgment *before* seeing the AI's score, and are scored over time on **calibration** (a Brier-score-style metric), not agreement.

How It Works (Technical Flow):

1. Extend the existing HSE Review screen: AI's band/explanation is hidden by default. Reviewer submits their own band + confidence first.
2. Reveal happens only after submission, followed by a mandatory reconciliation: agree, or override with a reason (fits straight into your existing `HSEReview.reason` field — no schema change).

3. Compute a rolling "Cassandra score" per reviewer once outcomes are confirmed — rewards being right *and* well-calibrated, not just occasionally correct.
4. A rotating **Challenger** role: once per shift, one reviewer is assigned to formally argue against the week's highest-confidence "critical" flag — a gamified pre-mortem.

Why Judges/OIL Will Love It: This is the single most concrete answer to the hardest question in your own judge-prep material — "what stops staff from just trusting the number?" — backed by citable HCI and decision-science research instead of a vague human-in-the-loop claim. It's also a UX pattern literally no EHS platform on the market has: you're not just building software, you're training better forecasters.

Hackathon Implementation Plan: The cheapest of the four. Hide-then-reveal is a UI state toggle on a screen you're already building. The leaderboard is one SQL aggregate over tables you already have, plus one new `outcome_band` field. Mock 8–10 "resolved" cases and this demo lives in 90 seconds.

If you only have time for two

#3 (Digital Bow-Tie) and **#1 (SIF-ATT&CK)** are your highest-leverage picks: they're the cheapest to build, the hardest for a judge to dismiss as "another LLM wrapper," and they directly upgrade the two things your architecture already leans on (the scoring formula and the taxonomy) rather than bolting on something separate. Add **#4** if you have even a spare afternoon — it's nearly free and it's the best possible answer to a question you *will* get asked.